Network Intrusion Detection with Classical Machine Learning

Project Report

Binary and Multiclass Classification on NSL-KDD

Hamza Abdelkader

March 19, 2026

1 Introduction

This project studies intrusion detection on the NSL-KDD dataset using a set of classical supervised learning models. NSL-KDD is a widely used benchmark for network security because it combines numerical and categorical features, strong class imbalance, and several attack families with very different frequencies.

The analysis covers two related tasks:

- **Binary classification:** distinguish normal traffic from malicious traffic.
- **Multiclass classification:** predict one of five classes: Normal, DoS, Probe, R2L, or U2R.

Studying both settings makes it possible to compare how the same models behave when the output space becomes more complex. It also highlights how class imbalance affects evaluation, especially for rare categories such as R2L and U2R.

2 Project Goals

The main goals of the project are:

- build a consistent preprocessing pipeline for mixed numerical and categorical data;
- compare several standard machine learning models on the same dataset;
- tune important hyperparameters with stratified cross-validation;
- evaluate generalization with accuracy, macro-F1, balanced accuracy, confusion matrices, ROC curves, and AUC;
- interpret the results through model assumptions, bias, variance, and class imbalance.

3 Dataset

The experiments use the NSL-KDD intrusion detection dataset, a refined version of the older KDD'99 benchmark. Each record describes a network connection through 41 features: 38 numerical attributes and 3 categorical attributes (`protocol_type`, `service`, and `flag`).

Two label mappings were considered:

- **Binary setup:** all attack types were merged into a single *Attack* class.
- **Multiclass setup:** attacks were grouped into the standard families *DoS*, *Probe*, *R2L*, and *U2R*, with *Normal* kept as a fifth class.

The binary training set contains 125,973 samples and the binary test set contains 22,544 samples. The multiclass problem is strongly imbalanced, which is especially visible for R2L and U2R.

No explicit rebalancing method such as oversampling, undersampling, or SMOTE was applied. This keeps the evaluation focused on how the models behave under the natural data distribution.

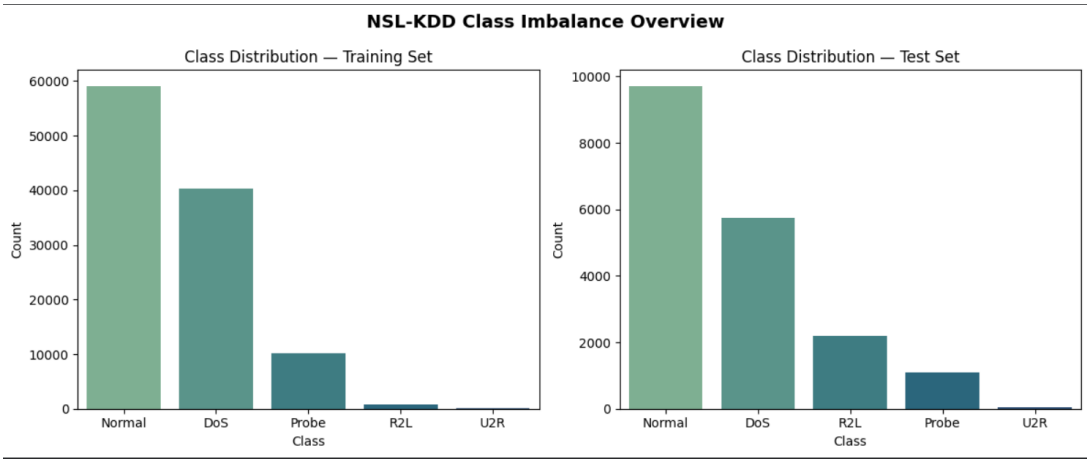


Figure 1: Class distribution in the NSL-KDD training and test sets for the multiclass setting.

4 Methodology

4.1 Preprocessing

The preprocessing pipeline treats numerical and categorical features separately. Numerical features are standardized using

$$z = \frac{x - \mu}{\sigma},$$

while categorical features are one-hot encoded. The transformation is applied through a unified pipeline so that fitting happens only on the training split.

```
preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat",
         OneHotEncoder(handle_unknown="ignore"),
         categorical_features)
    ]
)
```

This design avoids data leakage, supports unseen categorical values at test time, and keeps the workflow reproducible.

4.2 Models

The following models were compared:

- **Naive Bayes:** a probabilistic model with strong conditional independence assumptions.
- **Logistic Regression:** a linear classifier for the binary task.
- **Softmax Regression:** the multiclass extension of logistic regression.

- **Decision Tree:** a nonlinear classifier built from recursive impurity-based splits.
- **Random Forest:** an ensemble of decorrelated trees designed to reduce variance.

For the linear models, the decision rule is based on the logistic or softmax probabilities:

$$P(y = 1 \mid x) = \sigma(w^\top x),$$

$$P(y = k \mid x) = \frac{\exp(w_k^\top x)}{\sum_j \exp(w_j^\top x)}.$$

Decision Trees use impurity reduction, typically measured with the Gini index:

$$G = \sum_k p_k(1 - p_k).$$

4.3 Model Selection and Evaluation

Hyperparameters were tuned with 5-fold stratified cross-validation. This preserves class proportions across folds, which is especially important for the imbalanced multiclass task.

The evaluation emphasizes both overall performance and minority-class behavior:

- accuracy;
- precision, recall, and F1-score;
- macro-F1;
- balanced accuracy;
- confusion matrices;
- ROC curves and AUC.

5 Binary Classification Results

5.1 Cross-Validation and Test Accuracy

Model	Best CV Score	Test Accuracy
Logistic Regression	0.9965	0.7581
Decision Tree	0.9993	0.7822
Naive Bayes	0.9798	0.5646
Random Forest	0.9999	0.7648

Table 1: Binary classification performance: cross-validation versus test accuracy.

Observation: cross-validation scores are extremely high for all models, especially the tree-based ones. The drop on the held-out test set suggests that the dataset is easy to fit in-sample but substantially harder to generalize on unseen traffic.

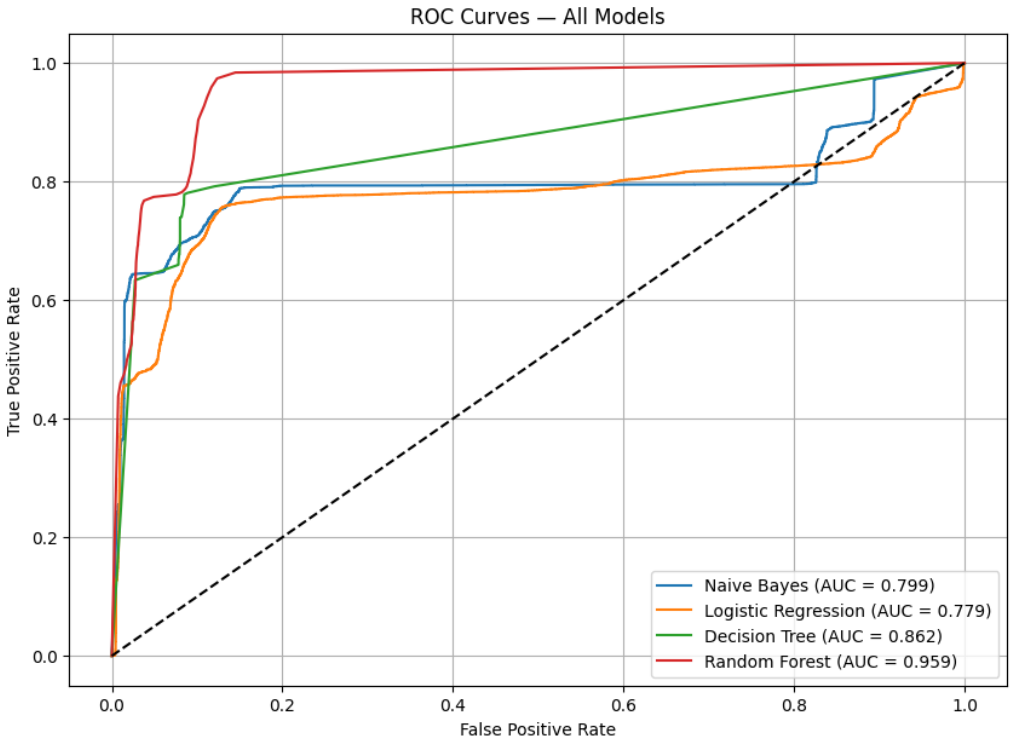


Figure 2: ROC curves for the binary classification task.

5.2 ROC Curves

Random Forest achieved the strongest binary ROC-AUC (0.959), followed by Decision Tree (0.862). The linear and probabilistic models showed weaker separation, which is consistent with their more restrictive assumptions.

5.3 Confusion Matrices

The confusion matrices show a clear ranking:

- **Naive Bayes** overpredicts the majority pattern and struggles to balance false positives and false negatives.
- **Logistic Regression** offers a solid tradeoff and benefits from stable convex optimization.
- **Decision Tree** captures nonlinear interactions more effectively and reaches the best binary macro-F1.
- **Random Forest** provides the best probability ranking and the most balanced overall performance.

The reported binary macro-F1 scores are 0.76 for Logistic Regression, 0.78 for Decision Tree, 0.76 for Random Forest, and 0.52 for Naive Bayes.

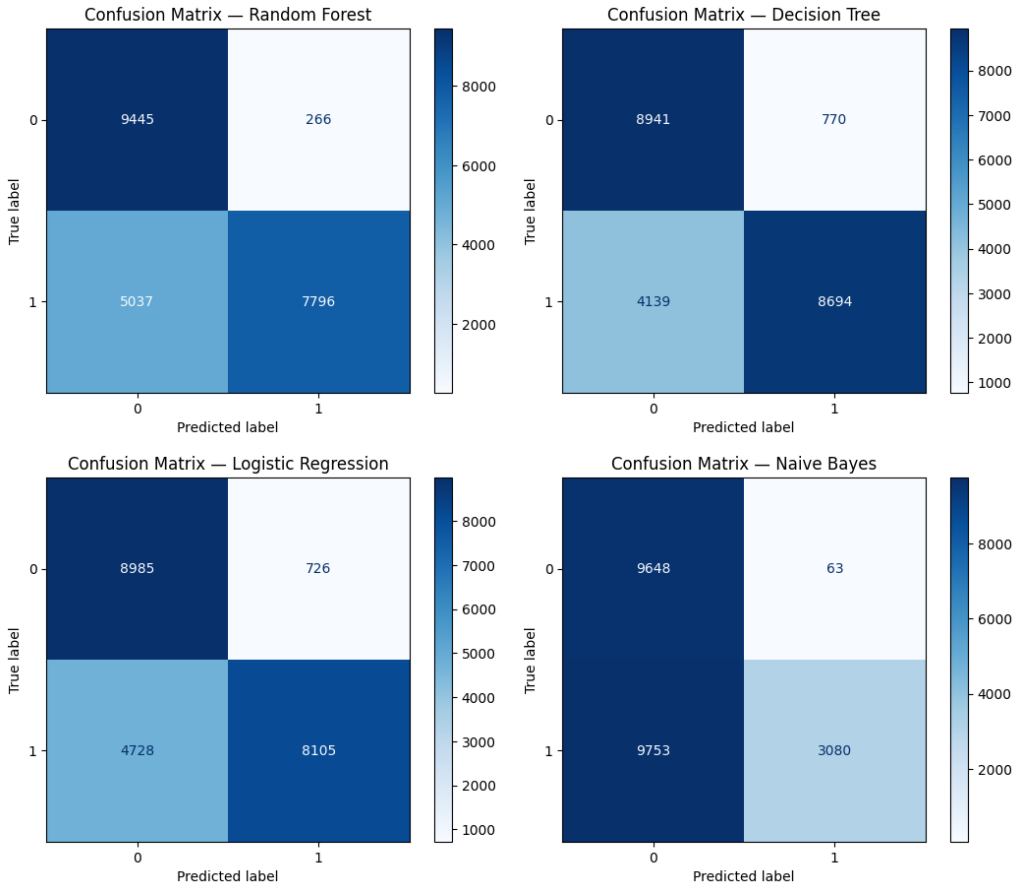


Figure 3: Confusion matrices for the binary classification task.

6 Multiclass Classification Results

6.1 Cross-Validation and Test Performance

Model	CV F1 (macro)	Test Accuracy	Test F1 (macro)	Balanced Accuracy
Naive Bayes	0.4333	0.5899	0.4097	0.5087
Softmax Regression	0.8581	0.8447	0.6499	0.6612
Decision Tree	0.9068	0.8445	0.5941	0.6211
Random Forest	0.9158	0.8803	0.6759	0.6765

Table 2: Multiclass classification performance on the NSL-KDD test set.

Random Forest produced the strongest multiclass results overall, while Softmax Regression remained highly competitive. The macro-F1 scores are more informative than accuracy here because they penalize failures on minority classes.

6.2 ROC Curves

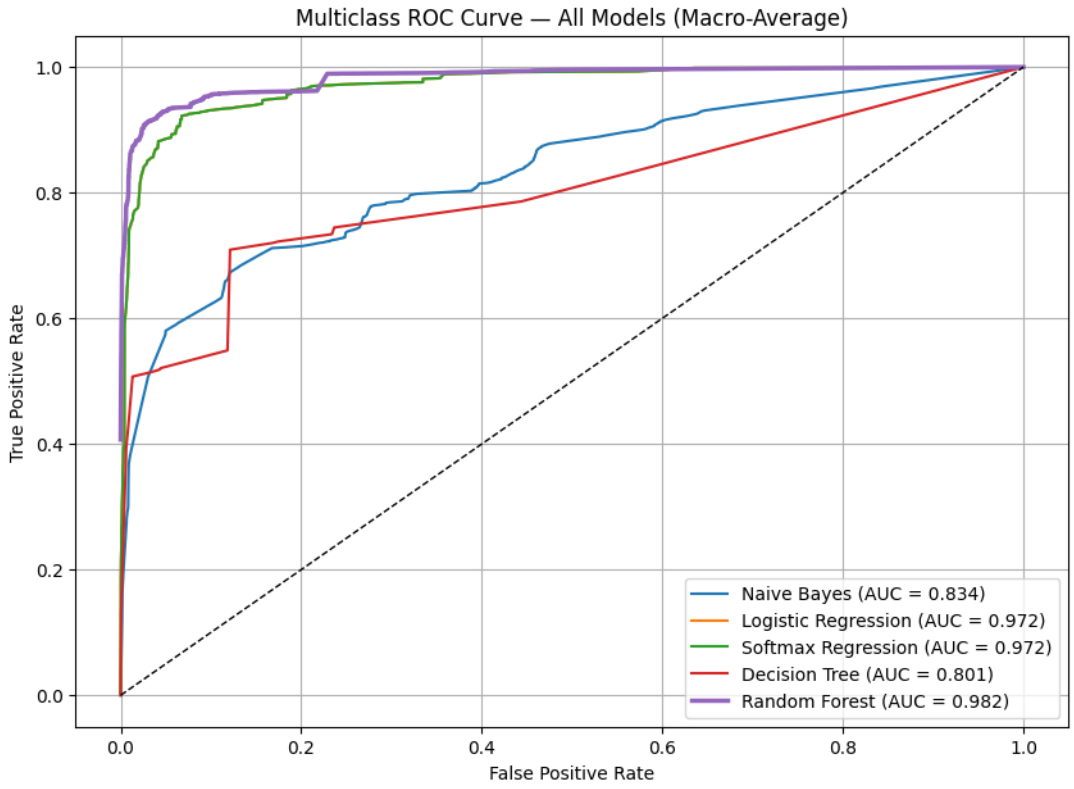


Figure 4: Macro-averaged ROC curves for the multiclass classification task.

Random Forest achieved the strongest macro-averaged AUC (0.982), indicating the best overall class separation.

6.3 Confusion Matrices

The confusion matrices reinforce the main challenges of the multiclass setting:

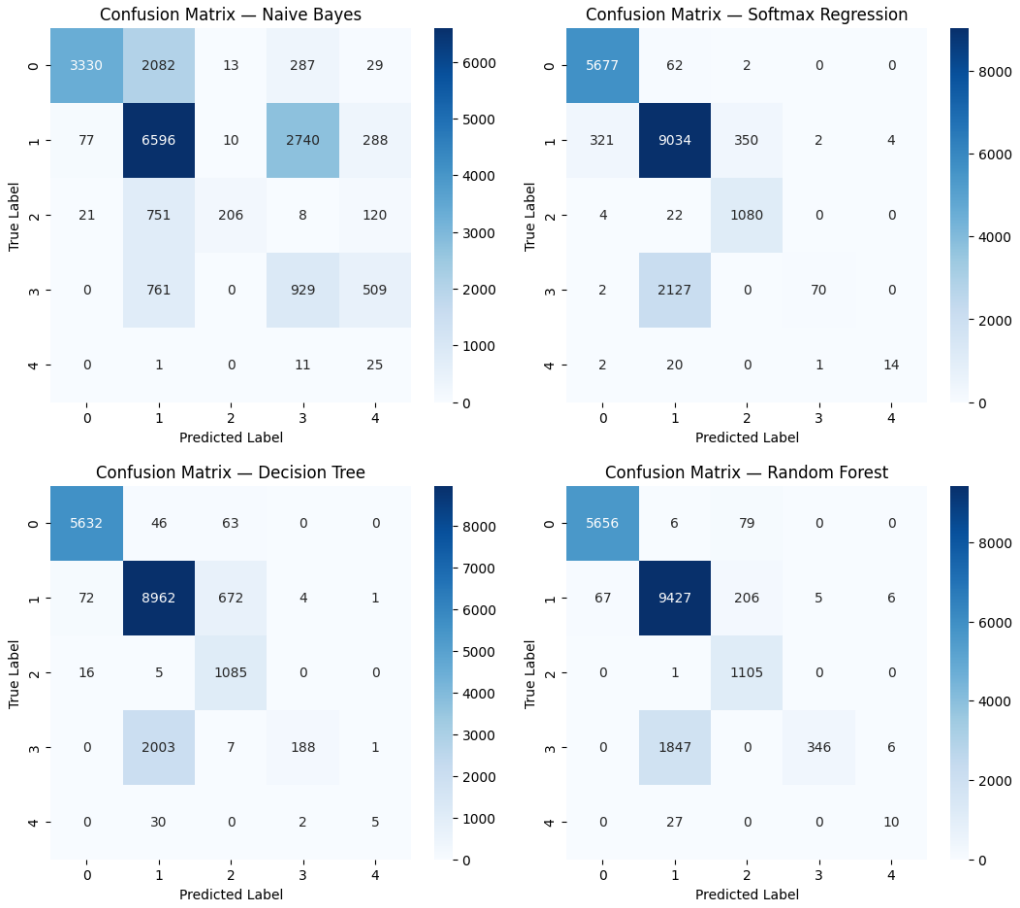


Figure 5: Confusion matrices for the multiclass classification task.

- **Naive Bayes** performs poorly on overlapping and minority classes.
- **Softmax Regression** is stable and competitive but still struggles on rare classes.
- **Decision Tree** fits frequent classes well but loses robustness on minority categories.
- **Random Forest** provides the best overall robustness, though rare-class prediction remains difficult.

7 Comparative Analysis

7.1 Binary vs. Multiclass Difficulty

The binary task is substantially easier than the multiclass task. In the binary setup, most models achieve reasonable test accuracy because they only need to separate benign and malicious traffic. In the multiclass setup, the models must separate five classes simultaneously, including two extremely rare classes. This increases overlap, amplifies the effect of label imbalance, and makes evaluation much more sensitive to model assumptions.

7.2 Effect of Model Assumptions

Each model reflects a different bias–variance profile:

- **Naive Bayes** has high bias and low variance. Its independence assumption is too restrictive for NSL-KDD, which explains its weak performance.
- **Logistic and Softmax Regression** have moderate bias and stable optimization. They work well when the transformed feature space is close to linearly separable, but they miss some nonlinear attack structure.
- **Decision Tree** has low bias but high variance. It can fit the training data extremely well and still generalize poorly.
- **Random Forest** reduces tree variance through bagging and random feature selection, which explains its superior overall performance on both tasks.

7.3 Generalization

The gap between cross-validation and test performance is especially informative for the tree-based methods. Decision Tree and Random Forest reach near-perfect cross-validation scores in the binary task and very high cross-validation macro-F1 in the multiclass task, but both decline on the held-out test set. This pattern strongly suggests overfitting pressure, even after hyperparameter tuning.

Softmax Regression shows a smaller generalization gap, which is consistent with its convex optimization and stronger regularization. Naive Bayes shows relatively smaller gaps as well, but mostly because its ceiling is already low.

7.4 Class Imbalance

Class imbalance is the defining difficulty of the multiclass problem. Accuracy alone hides the fact that most models struggle on R2L and U2R. Metrics such as macro-F1 and balanced accuracy make these weaknesses visible. Random Forest is the most robust model under imbalance, but even it does not fully solve the rare-class problem.

8 Conclusion

This project shows that classical machine learning models remain effective for tabular intrusion detection, especially when combined with careful preprocessing and cross-validation.

Random Forest emerged as the strongest model overall. It delivered the best binary ROC-AUC, the best multiclass accuracy, the best multiclass macro-F1, and the best balanced accuracy. Softmax Regression was also a strong baseline, particularly in the multiclass setting, thanks to its stable optimization and good generalization. Decision Trees worked well in the binary task but showed clearer signs of high variance. Naive Bayes underperformed consistently because its independence assumptions were too restrictive for this dataset.

Future improvements could focus on class-imbalance handling, cost-sensitive learning, threshold tuning, or more advanced ensemble methods. Even without those extensions, the current results provide a useful comparison of how standard models behave on a challenging intrusion-detection benchmark.